Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#07)

Implementation of Arithmetic Logic Unit (ALU)

Student Name: Manahil Shahzad Roll No: 24I-6563 Section: CE-A

Date performed: 6th Oct, 2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____
Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

Task 1: Using the given design, write a Verilog code at all level of abstraction to make a Module for 1 bit ALU having inputs (1 bit) as A and B as input data, carry_in, select line (2 bit input) to select between different operations i.e. AND, OR and ADD/SUB. The module will have an Output for Result of operations and another output for carry_out. Selection between ADD/SUB will be provided by another input b_inv. Show simulation results for every instruction executed. Make sure to design a circuit diagram using logisim and a truth table to verify the working of ALU.

Description:

This code implements a 1-bit ALU using data-flow, structural and behavioral modeling with logic gates such as AND, OR, XOR, and NOT. It performs AND, OR, addition, and subtraction operations based on the select inputs. The testbench applies different input combinations with time delays to observe the output.

Code:

Dataflow Modeling:

```
module alu1_dataflow(

    input a, b, cin, binv,
    input [1:0] sel,
    output result, cout
);

    wire bx, sum, carry;

    assign bx    = b ^ binv;

    assign sum   = a ^ bx ^ cin;
    assign carry = (a & bx) | (a & cin) | (bx & cin);

    assign result = (sel == 2'b00)?(a & b) :
                    (sel == 2'b01)?(a | b) :
                    (sel == 2'b10)?sum : 1'b0;

    assign cout = (sel == 2'b10) ? carry : 1'b0;

endmodule

module tb_alu1_dataflow;

reg a, b, cin, binv;
reg [1:0] sel;
```

```
wire result, cout;

alu1_dataflow uut(a, b, cin, binv, sel, result, cout);

initial
begin
   sel = 2'b00; binv = 0; cin = 0;
   a = 0; b = 0;
   #50 a = 0; b = 1;
   #50 a = 1; b = 0;
   #50 a = 1; b = 1;


   #50 sel = 2'b01; a = 0; b = 0;
   #50 a = 0; b = 1;
   #50 a = 1; b = 0;
   #50 a = 1; b = 1;


   #50 sel = 2'b10; binv = 0; cin = 0;
   a = 0; b = 0;
   #50 a = 0; b = 1;
   #50 a = 1; b = 0;
   #50 a = 1; b = 1;


   #50 sel = 2'b10; binv = 1; cin = 1;
   a = 0; b = 0;
   #50 a = 0; b = 1;
   #50 a = 1; b = 0;
   #50 a = 1; b = 1;


   #50 sel = 2'b11; a = 1; b = 1;

   #50 $finish;
end

endmodule
```

Structural Modeling:

```
module full_adder_gl(
   input a, b, cin,
   output sum, cout
);
```

```
    wire x1, t1, t2;

    xor (x1, a, b);
    xor (sum, x1, cin);
    and (t1, a, b);
    and (t2, x1, cin);
    or  (cout, t1, t2);

endmodule


module mux4_gl(
    input d0, d1, d2, d3,
    input s1, s0,
    output y
);

    wire ns1, ns0, w0, w1, w2, w3;

    not (ns1, s1);
    not (ns0, s0);

    and (w0, d0, ns1, ns0);
    and (w1, d1, ns1, s0);
    and (w2, d2, s1, ns0);
    and (w3, d3, s1, s0);

    or (y, w0, w1, w2, w3);

endmodule


module alu1_structural(
    input a, b, cin, binv,
    input [1:0] sel,
    output result, cout
);

    wire nb, nbinv, t1, t2;
    wire bx, sum, carry_raw;
    wire and_o, or_o;


    not (nb, b);
    not (nbinv, binv);
    and (t1, b, nbinv);
```

```
    and (t2, nb, binv);
    or  (bx, t1, t2);


    and (and_o, a, b);
    or  (or_o, a, b);


    full_adder_gl fa(.a(a), .b(bx), .cin(cin),.sum(sum), .cout(carry_raw));

    mux4_gl mux(
        .d0(and_o), .d1(or_o),
        .d2(sum), .d3(1'b0),
        .s1(sel[1]), .s0(sel[0]),
        .y(result)
    );


    assign cout = (sel == 2'b10) ? carry_raw : 1'b0;

endmodule

module tb_alu1_structural;

reg a, b, cin, binv;
reg [1:0] sel;
wire result, cout;

alu1_structural uut(a, b, cin, binv, sel, result, cout);

initial
begin

    sel = 2'b00; binv = 0; cin = 0;
    a = 0; b = 0;
    #50 a = 0; b = 1;
    #50 a = 1; b = 0;
    #50 a = 1; b = 1;


    #50 sel = 2'b01; a = 0; b = 0;
    #50 a = 0; b = 1;
    #50 a = 1; b = 0;
    #50 a = 1; b = 1;


    #50 sel = 2'b10; binv = 0; cin = 0;
```

```
    a = 0; b = 0;
    #50 a = 0; b = 1;
    #50 a = 1; b = 0;
    #50 a = 1; b = 1;


    #50 sel = 2'b10; binv = 1; cin = 1;
    a = 0; b = 0;
    #50 a = 0; b = 1;
    #50 a = 1; b = 0;
    #50 a = 1; b = 1;

    #50 sel = 2'b11; a = 1; b = 1;

    #50 $finish;
end

endmodule
```

Behavioural Modeling:

```
module alu1_behavioral(
    input a, b, cin, binv,
    input [1:0] sel,
    output reg result, cout
);

    reg bx;
    reg [1:0] arithmetic;

    always @(*) begin

        bx = b ^ binv;
        arithmetic = {1'b0, a} + {1'b0, bx} + cin;

        result = 1'b0;
        cout   = 1'b0;

        case (sel)
            2'b00: result = a & b;
            2'b01: result = a | b;

            2'b10: begin
                result = arithmetic[0];
                cout   = arithmetic[1];
            end
```

```
        2'b11: begin
            result = 1'b0;
            cout   = 1'b0;
          end
      endcase

  end

endmodule
module tb_alu1_behavioral;

reg a, b, cin, binv;
reg [1:0] sel;
wire result, cout;

alu1_behavioral uut(a, b, cin, binv, sel, result, cout);

initial
begin

  sel = 2'b00; binv = 0; cin = 0;
  a = 0; b = 0;
  #50 a = 0; b = 1;
  #50 a = 1; b = 0;
  #50 a = 1; b = 1;

  #50 sel = 2'b01; a = 0; b = 0;
  #50 a = 0; b = 1;
  #50 a = 1; b = 0;
  #50 a = 1; b = 1;


  #50 sel = 2'b10; binv = 0; cin = 0;
  a = 0; b = 0;
  #50 a = 0; b = 1;
  #50 a = 1; b = 0;
  #50 a = 1; b = 1;


  #50 sel = 2'b10; binv = 1; cin = 1;
  a = 0; b = 0;
  #50 a = 0; b = 1;
  #50 a = 1; b = 0;
  #50 a = 1; b = 1;
  #50 sel = 2'b11; a = 1; b = 1;

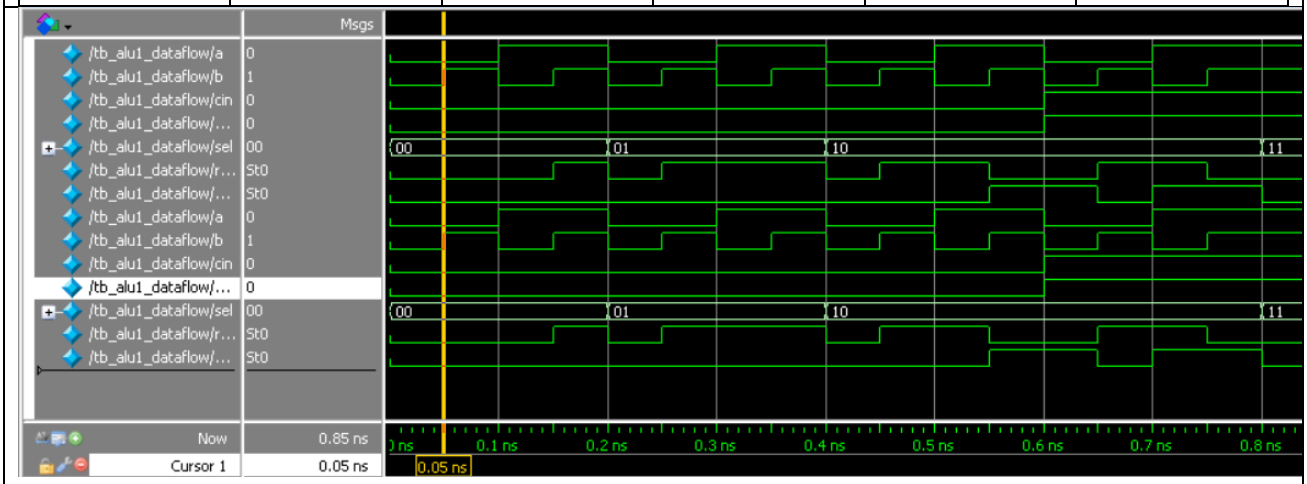
  #50 $finish;
```

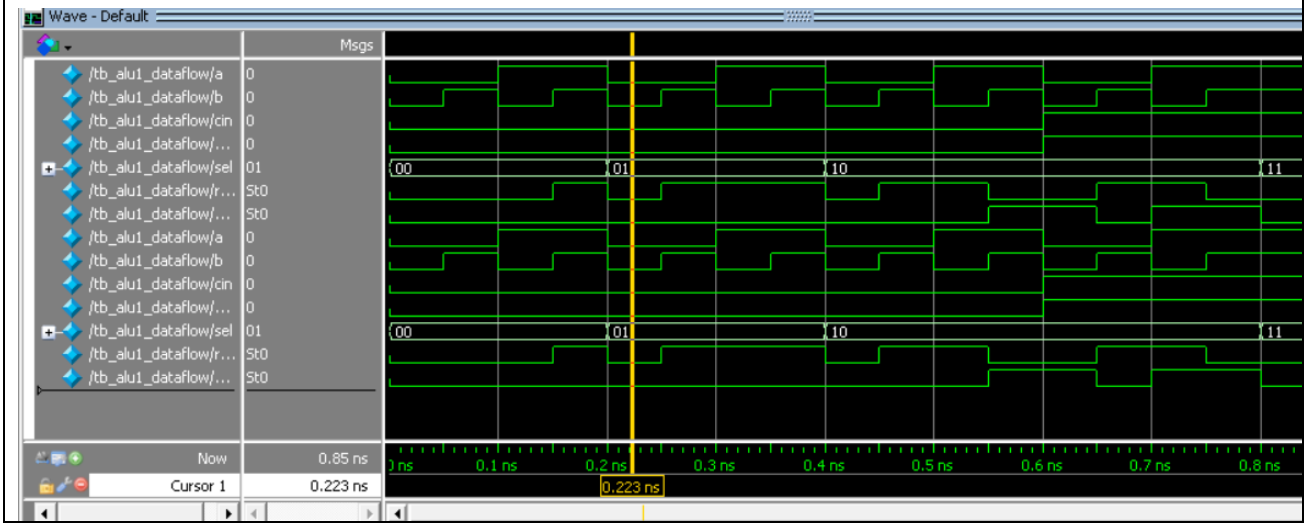
end

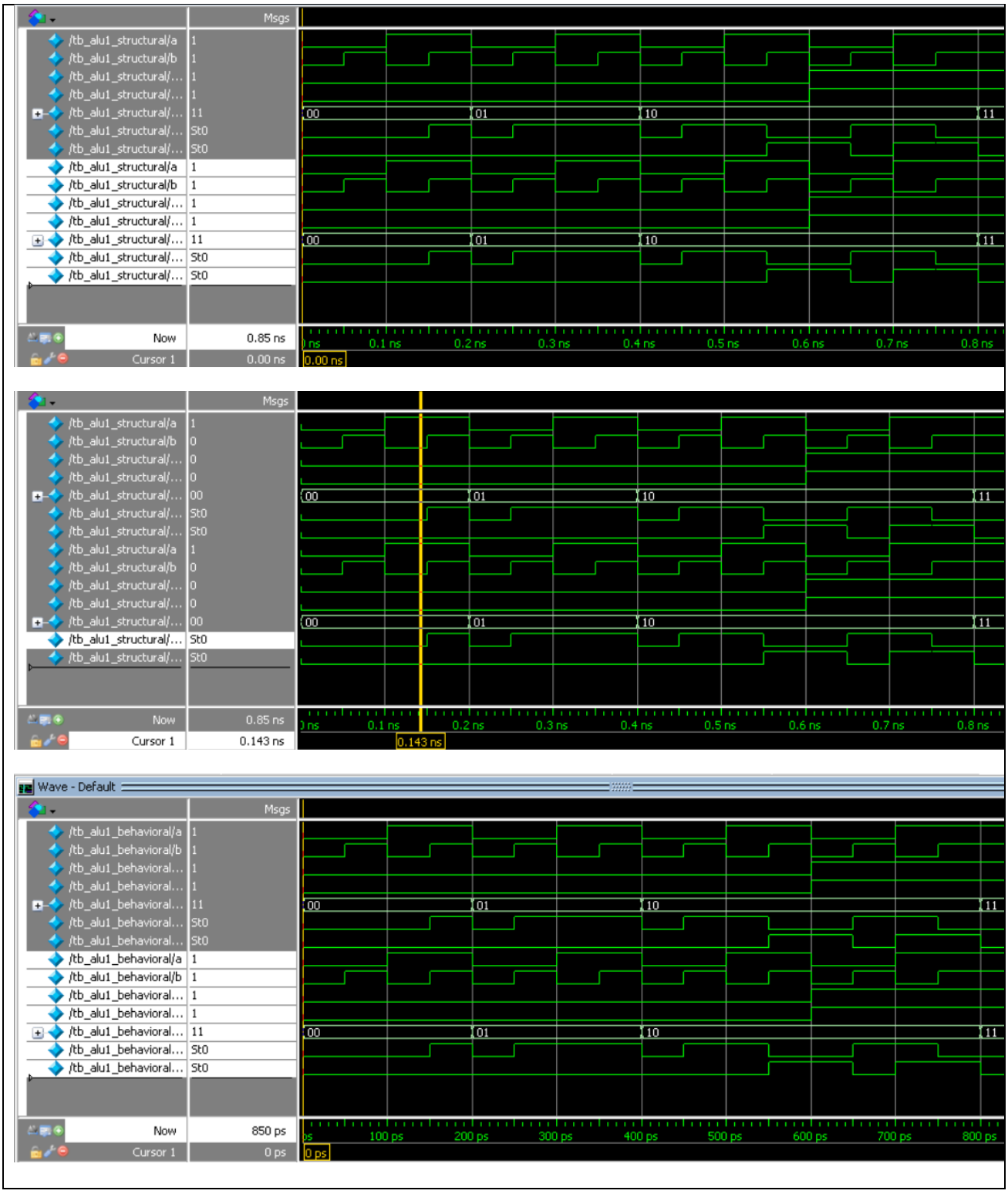
endmodule

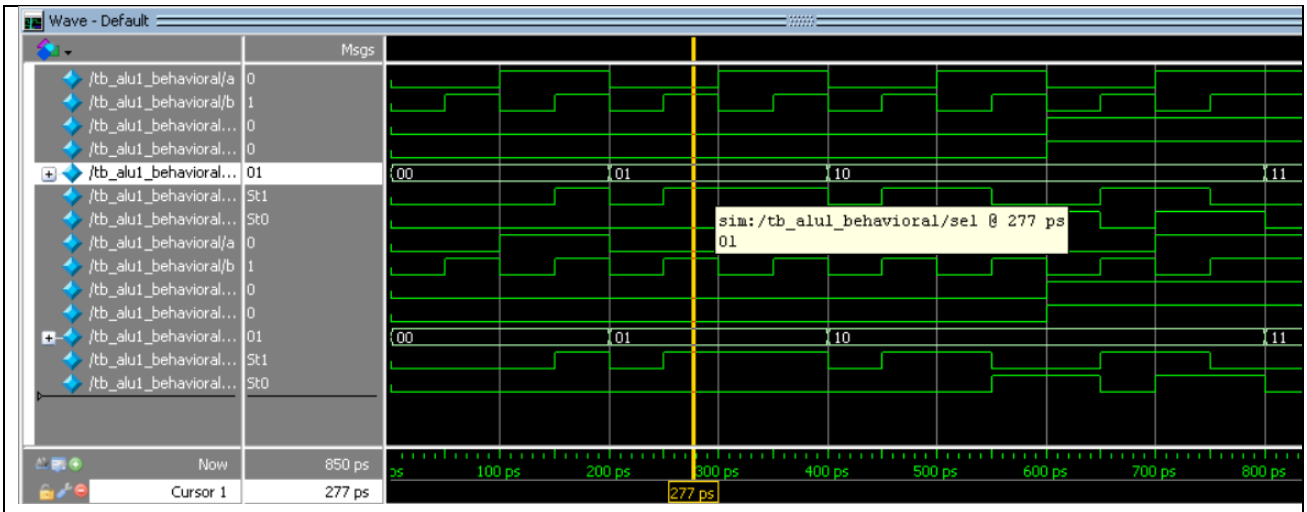
TRUTH TABLE:

Sel 1	Sel 0	Bin	Cin	A	B	Cout
0	0	0	0	1	1	1
0	1	0	0	0	1	0
1	0	0	0	1	1	1
1	0	1	1	1	1	1
1	0	1	1	0	1	0

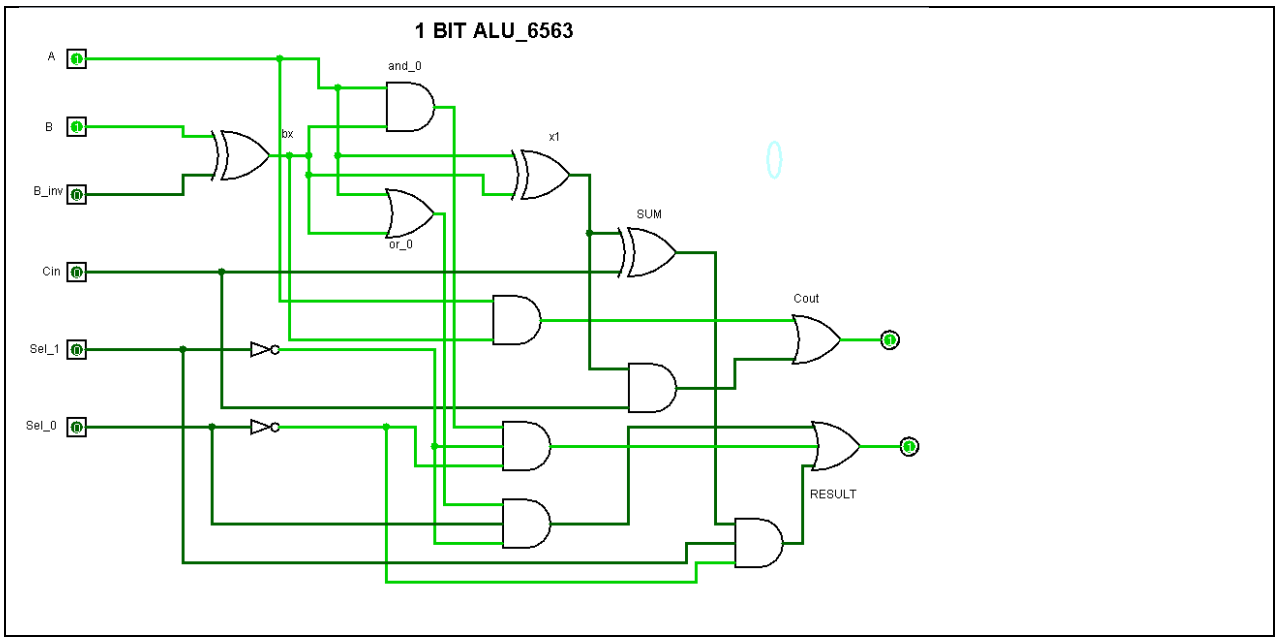


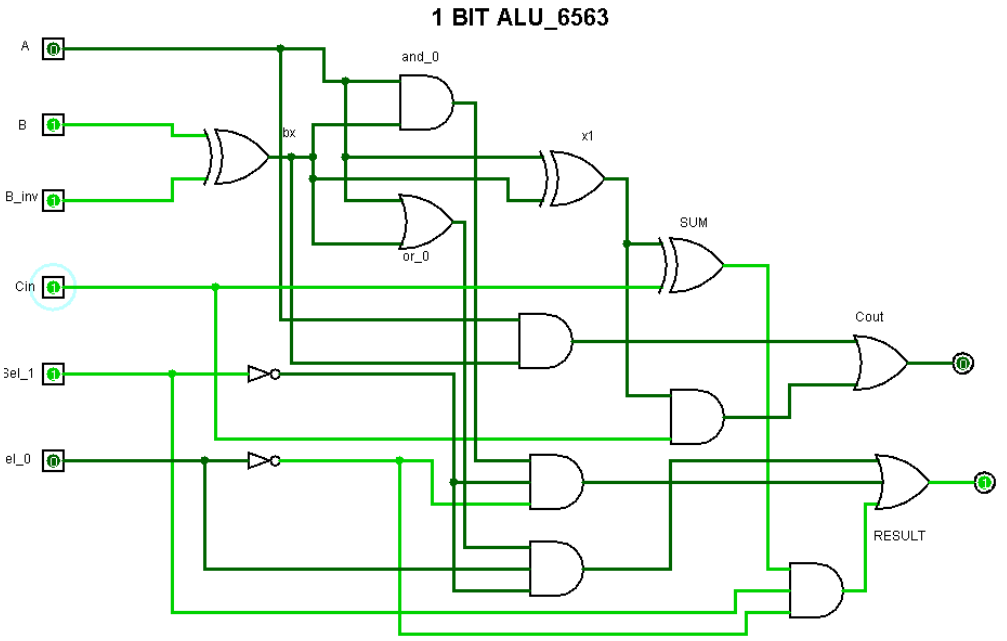


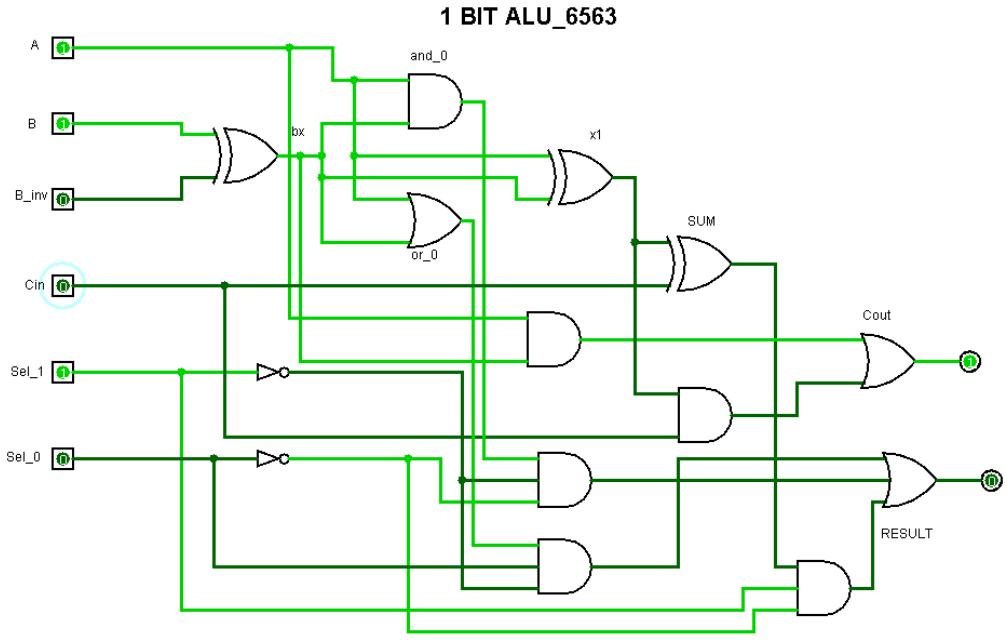

Msgs
/tb_alu1_structural/a
/tb_alu1_structural/b
/tb_alu1_structural/...
/tb_alu1_structural/...
/tb_alu1_structural/...
/tb_alu1_structural/...
/tb_alu1_structural/...
/tb_alu1_structural/a
/tb_alu1_structural/b
/tb_alu1_structural/...
/tb_alu1_structural/...
/tb_alu1_structural/...
/tb_alu1_structural/...
/tb_alu1_structural/...
Now
0.85 ns
Cursor 1
0.00 ns
0.1 ns
0.2 ns
0.3 ns
0.4 ns
0.5 ns
0.6 ns
0.7 ns
0.8 ns
00
01
10
11
St0
Msgs
Now
0.85 ns
Cursor 1
0.143 ns
Wave - Default
Msgs
/tb_alu1_behavioral/a
/tb_alu1_behavioral/b
/tb_alu1_behavioral...
/tb_alu1_behavioral...
/tb_alu1_behavioral...
/tb_alu1_behavioral...
/tb_alu1_behavioral...
/tb_alu1_behavioral/a
/tb_alu1_behavioral/b
/tb_alu1_behavioral...
/tb_alu1_behavioral...
/tb_alu1_behavioral...
/tb_alu1_behavioral...
/tb_alu1_behavioral...
Now
850 ps
Cursor 1
0 ps
100 ps
200 ps
300 ps
400 ps
500 ps
600 ps
700 ps
800 ps



LOGISIM OUTPUT:




1 BIT ALU_6563
A
B
B_inv
Cin
Sel_1
Sel_0
bx
and_0
or_0
x1
SUM
Cout
RESULT


1 BIT ALU_6563
A
B
B_inv
Cin
Sel_1
Sel_0
bx
and_0
or_0
x1
SUM
Cout
RESULT

Task 2: Write Verilog code to build a 4bit ALU out of 4 1bit ALUs. The ALU must be capable of performing ADD, SUB operations and logical operation that are AND and OR operations and a SLT operation. Status flags (CF, SF, ZF and OF) must also be set accordingly Test the code using test bench and simulation.

Description:

This code implements a 4-bit ALU using four cascaded 1-bit ALUs with dataflow modeling. It performs AND, OR, addition, subtraction, and signed SLT operations according to the select inputs. The testbench applies different input combinations to test the operations and verify the status flags CF, SF, ZF, and OF through simulation.

Code:

```
module alu1(
   input a, b, cin, binv, less,
   input [1:0] op,
   output result, cout, sum
);

wire bx;

assign bx = b ^ binv;

assign sum = a ^ bx ^ cin;
assign cout = (a & bx) | (a & cin) | (bx & cin);

assign result = (op == 2'b00) ? (a & b) :
            (op == 2'b01) ? (a | b) :
            (op == 2'b10) ? sum : less;

endmodule


module alu4(
   input [3:0] a, b,
   input binv,
   input [1:0] op,
   output [3:0] result,
   output cf, sf, zf, of
);

wire [4:0] c;
wire [3:0] sum;
wire sub, overflow, less;
```

```
assign sub = binv | (op == 2'b11);
assign c[0] = sub;

alu1 u0(a[0], b[0], c[0], sub, less,
      op, result[0], c[1], sum[0]);

alu1 u1(a[1], b[1], c[1], sub, 1'b0,
      op, result[1], c[2], sum[1]);

alu1 u2(a[2], b[2], c[2], sub, 1'b0,
      op, result[2], c[3], sum[2]);

alu1 u3(a[3], b[3], c[3], sub, 1'b0,
      op, result[3], c[4], sum[3]);

assign overflow = c[3] ^ c[4];
assign less = sum[3] ^ overflow;

assign cf = (op == 2'b10) ? c[4] : 1'b0;
assign of = (op == 2'b10) ? overflow : 1'b0;
assign sf = result[3];
assign zf = (result == 4'b0000);

endmodule


module tb_alu4;

reg [3:0] a, b;
reg binv;
reg [1:0] op;

wire [3:0] result;
wire cf, sf, zf, of;

alu4 uut(a, b, binv, op, result, cf, sf, zf, of);

initial
begin
   op = 2'b00; binv = 0;
   a = 4'b1100; b = 4'b1010;
   #50;

   op = 2'b01;
   #50;

   op = 2'b10; binv = 0;
```

```
    a = 4'b0011; b = 4'b0100;
    #50;

    a = 4'b1111; b = 4'b0001;
    #50;
    binv = 1;
    a = 4'b0101; b = 4'b0011;
    #50;

    a = 4'b0101; b = 4'b0101;
    #50;

    op = 2'b11; binv = 0;
    a = 4'b0011; b = 4'b0101;
    #50;

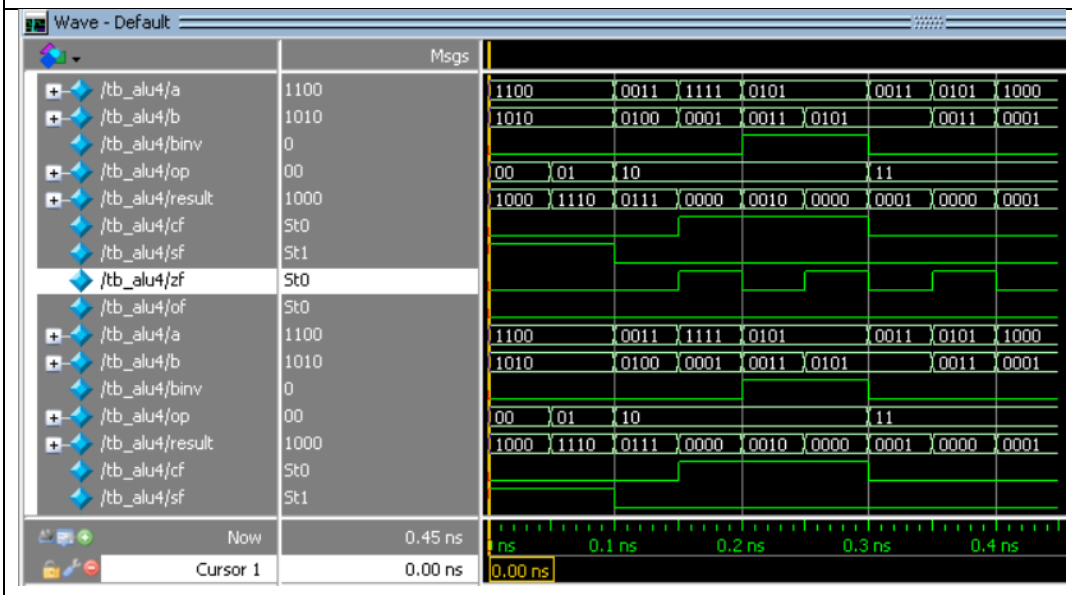
    a = 4'b0101; b = 4'b0011;
    #50;

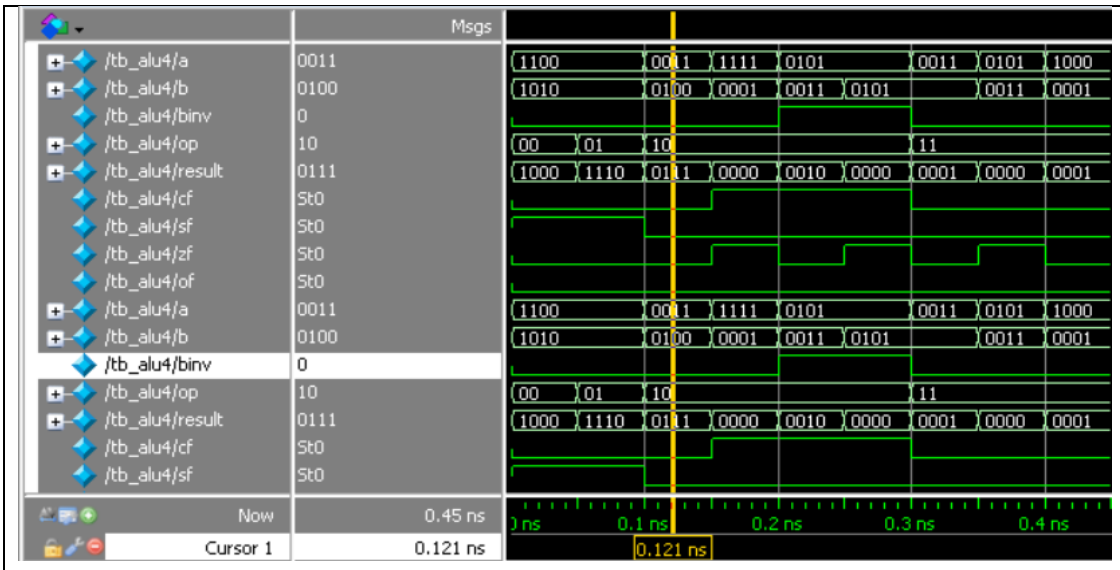
    a = 4'b1000; b = 4'b0001;
    #50;

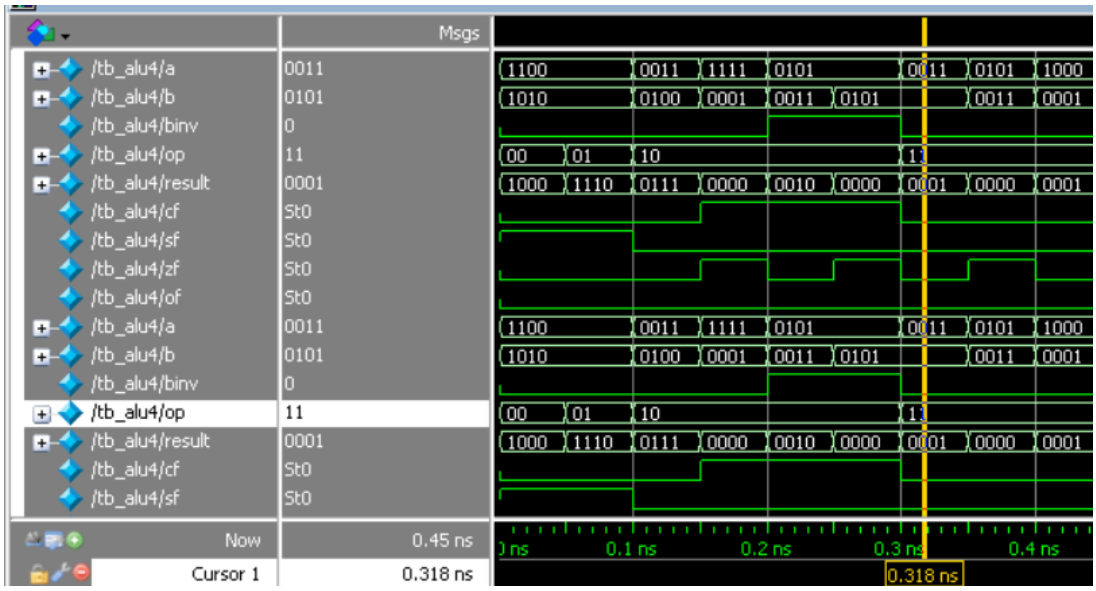
    $finish;
end

endmodule
```

Output:







Task 3:Write a Verilog code for a parameterized ALU, where the parameter N specifies the bit-width of the input and output signals. The operation of the ALU is determined by a separate control signal. The ALU should support the following operations:

a. Addition

b. Subtraction

c. Basic logical operations (AND, OR, NOT, XOR)

The inputs and outputs of the ALU must be N-bit wide, making the design scalable and reusable for different bit sizes.

Description:

This task involves designing a parameterized ALU in Verilog that can perform different operations depending on the control signal. The ALU supports addition, subtraction, AND, OR, NOT, and XOR operations. The parameter `N` allows the bit-width of the inputs and output to be changed as needed. A testbench is also used to check the results of each operation and verify the functionality of the ALU.

Code:

```
module alu_param #(parameter N = 8) (
   input [N-1:0] a, b,
   input [2:0] ctrl,
   output reg [N-1:0] result,
   output reg carry_out,
   output zero
);

always @(*) begin
   result = {N{1'b0}};
   carry_out = 1'b0;

   case (ctrl)
      3'b000: begin
         {carry_out, result} = {1'b0, a} + {1'b0, b};
      end

      3'b001: begin
         result = a - b;
         carry_out = (a < b);
      end

      3'b010: result = a & b;
      3'b011: result = a | b;
      3'b100: result = ~a;
      3'b101: result = a ^ b;

      default: begin
         result = {N{1'b0}};
         carry_out = 1'b0;
      end
   endcase
end

assign zero = (result == {N{1'b0}});

endmodule
```

```
module tb_alu_param;

parameter N = 4;

reg [N-1:0] a, b;
reg [2:0] ctrl;
wire [N-1:0] result;
wire carry_out, zero;

alu_param #(.N(N)) uut (
    .a(a),
    .b(b),
    .ctrl(ctrl),
    .result(result),
    .carry_out(carry_out),
    .zero(zero)
);

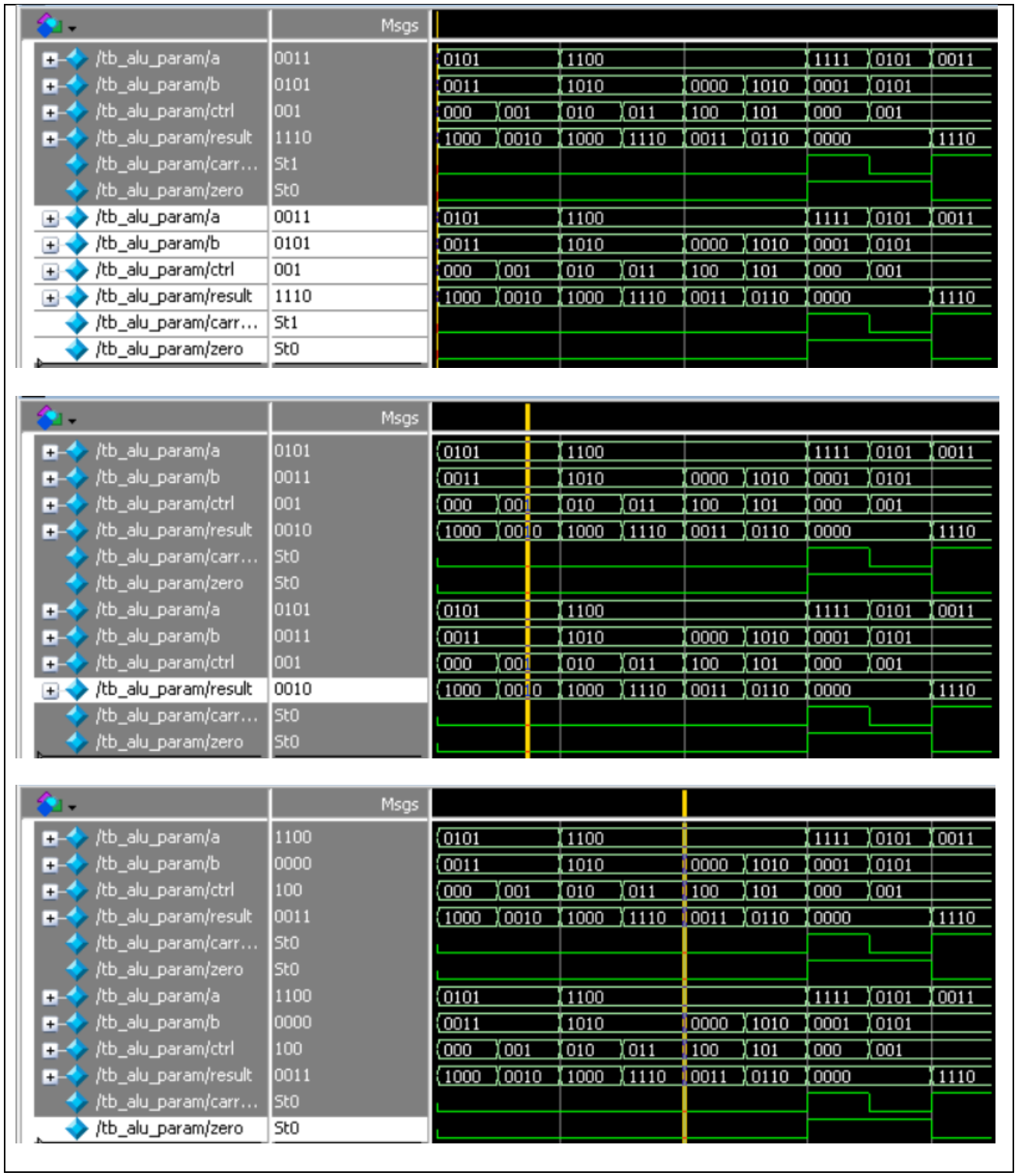
initial begin
    ctrl = 3'b000; a = 5;  b = 3;  #50;
    ctrl = 3'b001; a = 5;  b = 3;  #50;
    ctrl = 3'b010; a = 12; b = 10; #50;
    ctrl = 3'b011; a = 12; b = 10; #50;
    ctrl = 3'b100; a = 12; b = 0;  #50;
    ctrl = 3'b101; a = 12; b = 10; #50;

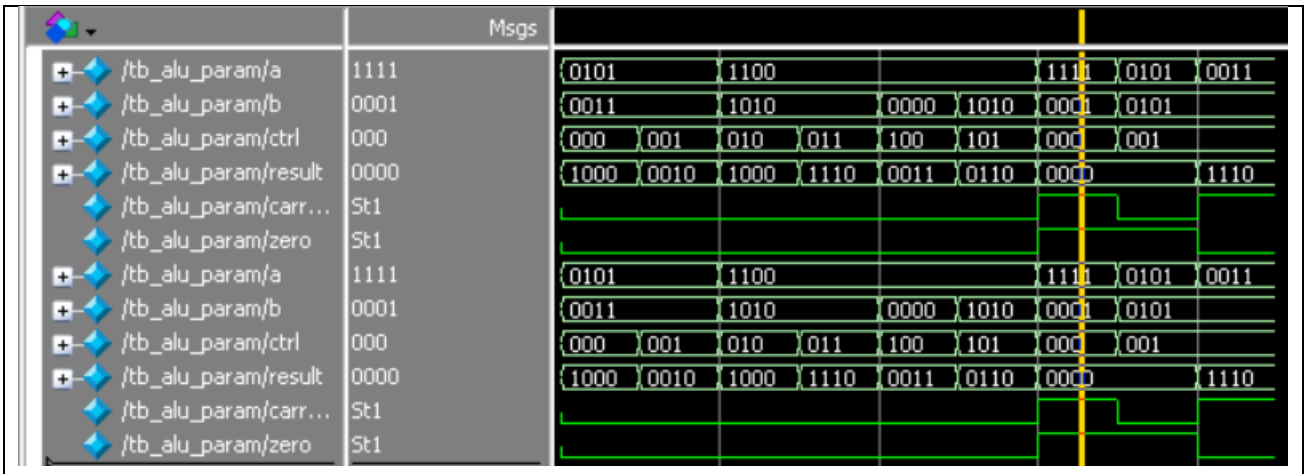
    ctrl = 3'b000; a = 15; b = 1;  #50;
    ctrl = 3'b001; a = 5;  b = 5;  #50;
    ctrl = 3'b001; a = 3;  b = 5;  #50;

end

endmodule
```

Output:


Msgs
/tb_alu_param/a
/tb_alu_param/b
/tb_alu_param/ctrl
/tb_alu_param/result
/tb_alu_param/carr...
/tb_alu_param/zero
0011
0101
001
1110
St1
St0
0101
0011
000
1000
1100
1010
001
0010
010
1000
011
1110
100
0000
101
0011
1010
0110
0000
0001
0101
000
0000
001
1110
1111
0101
0011



Task 4:

Description:
This task involves designing a fully functional 32-bit ALU in Verilog that performs arithmetic, logical, comparison, and shifting operations based on a 4-bit control signal. The ALU supports ten operations commonly used in RISC-V processors and generates a zero flag when the result is zero. A testbench is also created to test the different operations and check whether the ALU produces the expected results.
Code:

```
module alu32(
    //Input and Output Declarations
    input  [31:0] a, b,
    input  [3:0] alu_op,
    output reg [31:0] result,
    output zero
);

    //Signed Operands for Comparison and Shifting
    wire signed [31:0] sa = a;
    wire signed [31:0] sb = b;

    //ALU Operations
    always @(*) begin
        case (alu_op)

            // Arithmetic Operations
            4'b0000: result = a + b;
```

```
        4'b0001: result = a - b;

        // Logical Operations
        4'b0010: result = a & b;

        4'b0011: result = a | b;

        4'b0100: result = a ^ b;

        // Comparison Operations
        4'b0101: result = (sa < sb) ? 32'd1 : 32'd0;

        4'b0110: result = (a < b) ? 32'd1 : 32'd0;

        // Shift Operations
        4'b0111: result = a << b[4:0];

        4'b1000: result = a >> b[4:0];

        4'b1001: result = sa >>> b[4:0];

        // Default Operation
        default: result = 32'd0;

      endcase
   end

   // Part 4: Zero Flag
   assign zero = (result == 32'd0);

endmodule

module tb_alu32;

   //Testbench Signal Declarations
   reg [31:0] a, b;
   reg [3:0] alu_op;

   wire [31:0] result;
   wire zero;

   integer pass, fail;

   //Instantiate the ALU
   alu32 dut (.a(a),.b(b),.alu_op(alu_op),.result(result),.zero(zero));

   //Applying Inputs and Checking the Results
```

```
task check;
    input [3:0] op;
    input [31:0] x, y;
    input [31:0] expected;
    input exp_zero;

    begin
        alu_op = op;
        a = x;
        b = y;
        #10;

        if (result === expected && zero === exp_zero) begin
            pass = pass + 1;
            $display("PASS: op=%b a=%h b=%h result=%h zero=%b",
                    op, x, y, result, zero);
        end
        else begin
            fail = fail + 1;
            $display("FAIL: op=%b a=%h b=%h result=%h expected=%h zero=%b",
                    op, x, y, result, expected, zero);
        end
    end
endtask

// Part 4: Applying Test Cases
initial begin
    pass = 0;
    fail = 0;

    // Arithmetic Operations: ADD and SUB
    check(4'b0000, 32'd5, 32'd3, 32'd8, 0);
    check(4'b0000, 32'hFFFFFFFF, 32'd1, 32'd0, 1);
    check(4'b0001, 32'd10, 32'd3, 32'd7, 0);
    check(4'b0001, 32'd5, 32'd5, 32'd0, 1);

    // Logical Operations: AND, OR, XOR
    check(4'b0010, 32'hF0F0F0F0,
         32'h0FF00FF0, 32'h00F000F0, 0);

    check(4'b0011, 32'hF0F0F0F0,
         32'h0FF00FF0, 32'hFFF0FFF0, 0);

    check(4'b0100, 32'hFFFF0000,
         32'h0F0F0F0F, 32'hF0F00F0F, 0);

    // Comparison Operations: SLT and SLTU
```

```
        check(4'b0101, 32'hFFFFFFFF,
            32'd1, 32'd1, 0);

        check(4'b0110, 32'hFFFFFFFF,
            32'd1, 32'd0, 1);

        check(4'b0101, 32'd2, 32'd9, 32'd1, 0);

        // Shift Operations: SLL, SRL, SRA
        check(4'b0111, 32'd1, 32'd4, 32'd16, 0);

        check(4'b1000, 32'h80000000,
            32'd4, 32'h08000000, 0);

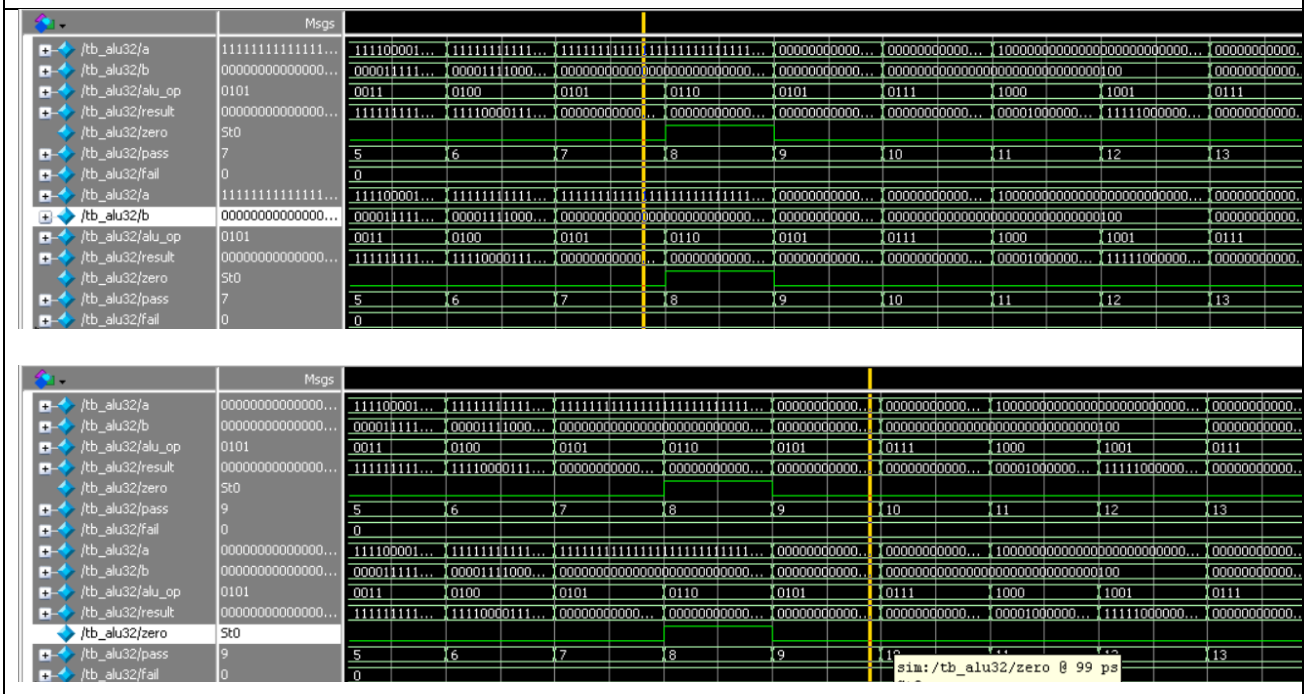
        check(4'b1001, 32'h80000000,
            32'd4, 32'hF8000000, 0);

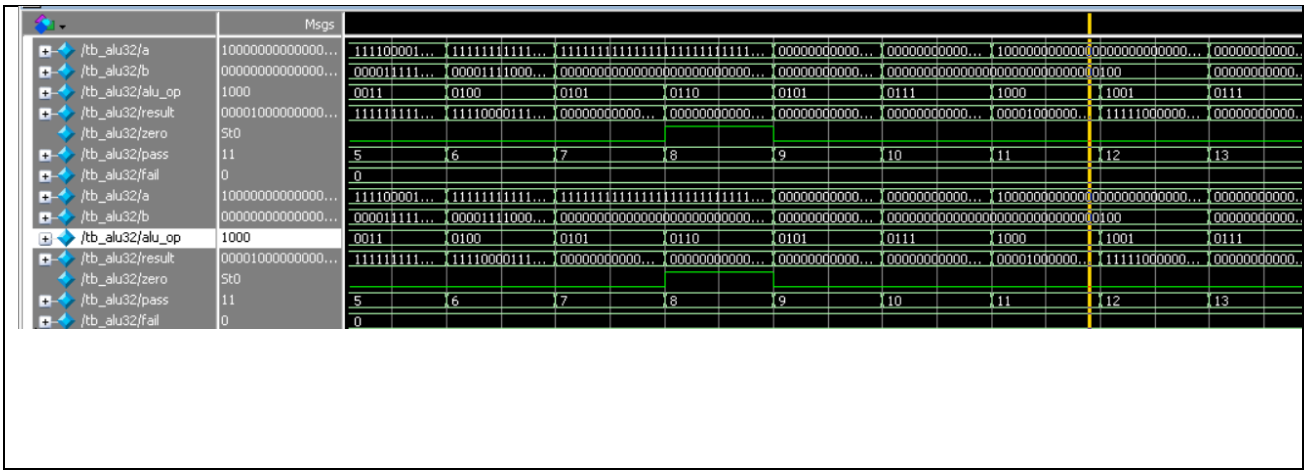
        // Edge Case: Shift Amount Uses Only Five Bits
        check(4'b0111, 32'd1, 32'd33, 32'd2, 0);

    end

endmodule
```

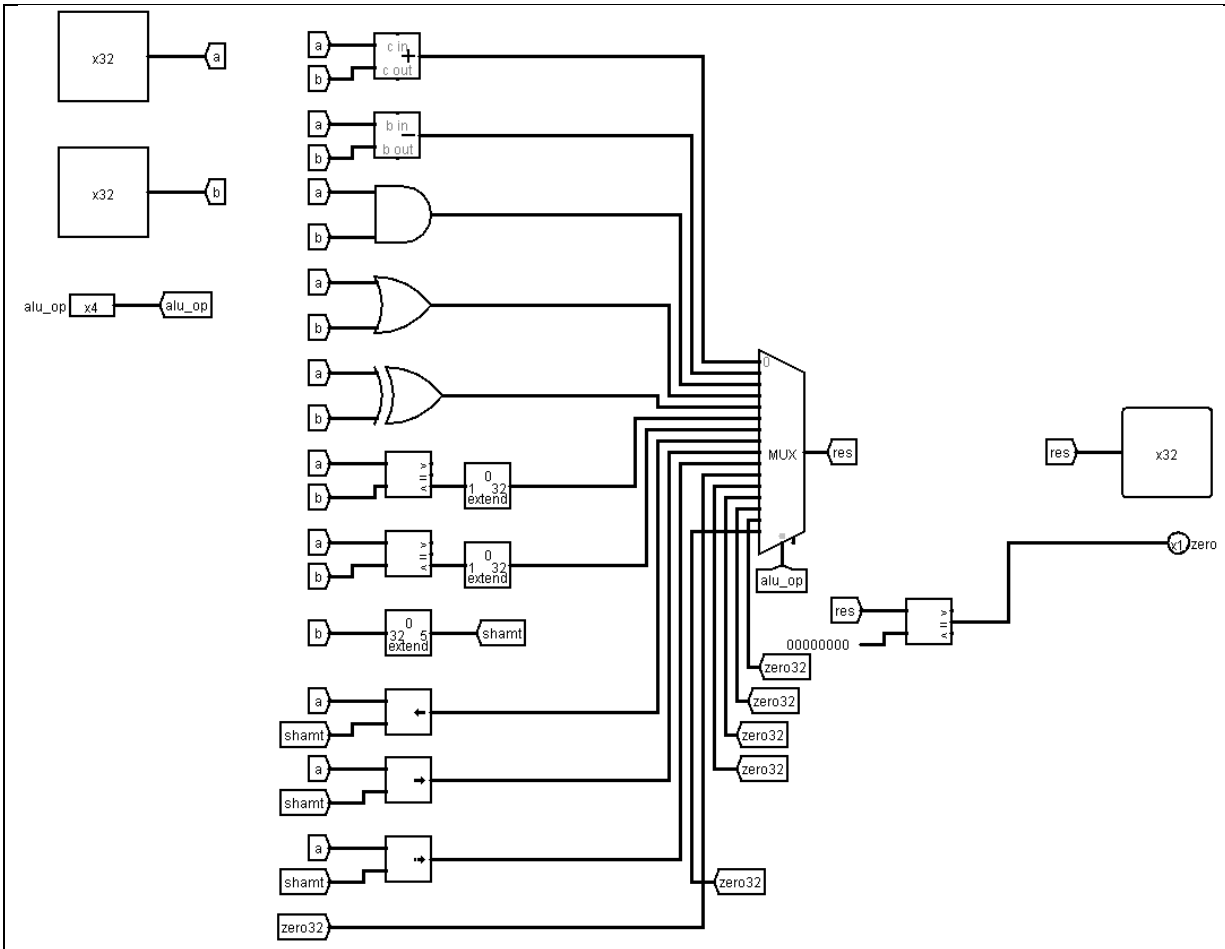
Output:





Task 5: Simulate the above (designed in task 4) ALU in Logisim.

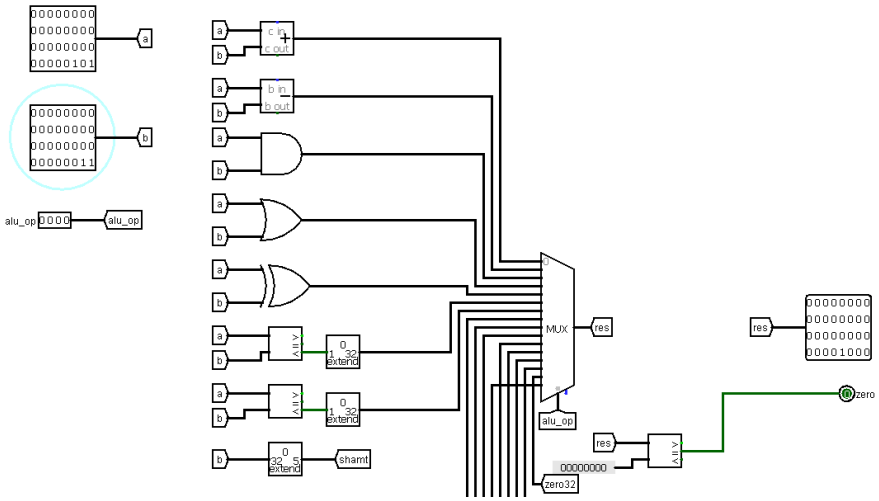
Description:
The objective of this task is to design and implement a fully functional 32-bit Arithmetic Logic Unit (ALU) in Logisim-evolution using built-in hardware components. t takes two 32-bit data inputs and uses a 4-bit control signal (alu_op) to select between 10 different arithmetic, logical, and shift operations.
Code:



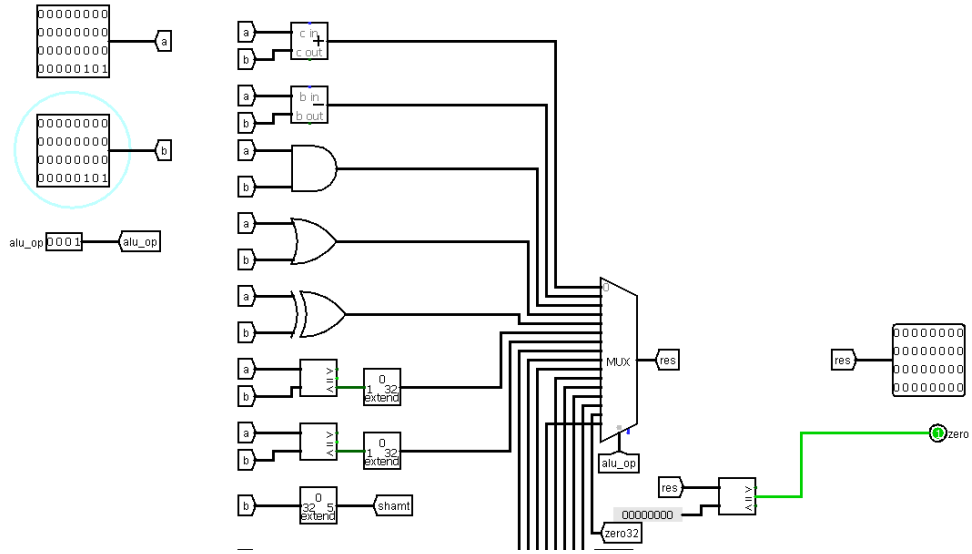
Output:

Truth Table:

alu_op	a	b	result	zero
0000 (ADD)	5	3	8	0
0000 (ADD)	FFFFFFFF	1	0	1
0001 (SUB)	A (10)	3	7	0
0001 (SUB)	5	5	0	1
0010 (AND)	F0F0F0F0	0FF00FF0	00F000F0	0
0011 (OR)	F0F0F0F0	0FF00FF0	FFF0FFF0	0
0100 (XOR)	FFFF0000	0F0F0F0F	F0F00F0F	0
0101 (SLT)	FFFFFFFF	1	1	0
0110 (SLTU)	FFFFFFFF	1	0	1
0111 (SLL)	1	4	10	0
1000 (SRL)	80000000	4	08000000	0
1001 (SRA)	80000000	4	F8000000	0



ADD=0000,a=5,b=3 and res=8



SUB=0001,a=5,b=5 res=0 and zero=1